Multi-UAV-Enabled Collaborative Edge Computing: Deployment, Offloading and Resource Optimization

Lin Tan, *Student Member, IEEE*, Songtao Guo, *Senior Member, IEEE*, Pengzhan Zhou, *Member, IEEE*, Zhufang Kuang, *Member, IEEE*, Saiqin Long, and Zhetao Li, *Senior Member, IEEE*

***Abstract*— Unmanned aerial vehicle (UAV) edge computing systems provide easy-to-deploy and low-cost services at those areas with inadequate infrastructure by deploying UAVs as moving edge servers for large-scale users. However, user devices are generally distributed unevenly in a large area, which makes it difficult for existing efforts to cope with this realistic scenario for optimal deployment of UAVs. Therefore, this paper considers a multiple UAV (Multi-UAV) Collaborative edge Computing (UCC) system by utilizing collaboration among them to split computation tasks at UAVs to balance the load and improve resource utilization. In order to maximize the energy-efficiency of the UCC system under the satisfaction of the delay constraint, we study the joint problem of UAV deployment, task collaborative offloading, computation and communication resource allocation in UCC system. We propose a bi-level optimization framework to solve the formulated non-convex mixed-integer optimization problem. In the upper level, the UAV deployment is optimized based on an improved differential evolution (DE) algorithm, and in the lower level the offloading decision and resource allocation are optimized based on a Reinforcement Learning (RL) algorithm with Twin Delayed Deep Deterministic policy gradient. Experimental results demonstrate the effectiveness and superiority of multi-UAV collaborative computing, with the proposed framework achieving a 32.4% reduction in energy consumption and an average 30% increase in task completion rate compared to DDPG, ToDeTaS, and other benchmark schemes.**

***Index Terms*— Edge computing, multi-UAV deployment, task offloading, resource allocation, differential evolution, deep reinforcement learning.**

Manuscript received 28 December 2023; revised 16 June 2024; accepted 17 July 2024. Date of publication 5 August 2024; date of current version 1 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272069, Grant 62202072, Grant 62203077, and Grant 62072477; in part by the Natural Science Key Foundation of Chongqing under Grant CSTB2024NSCQ-LZX0129; in part by the Fundamental Research Funds for the Central Universities under Grant 2023CDJYGRH-ZD06 and Grant 2021CDJXDJH006; in part by Hunan Provincial Natural Science Foundation of China under Grant 2024JJ5648; and in part by the National Key Research and Development Program of China under Grant 2023YFD2201703. The Associate Editor for this article was J. Blum. *(Corresponding authors: Songtao Guo; Pengzhan Zhou.)*

Lin Tan, Songtao Guo, and Pengzhan Zhou are with the Key Laboratory of Dependable Service Computing in Cyber-Physical-Society (Ministry of Education) and the College of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: guosongtao@cqu.edu.cn; pzzhou@cqu.edu.cn).

Zhufang Kuang is with the School of Computer and Information Engineering, Central South University of Forestry and Technology, Changsha 410004, China.

Saiqin Long and Zhetao Li are with the National Joint Engineering Research Center of Network Security Detection and Protection Technology, College of Information Science and Technology, Jinan University, Guangzhou 510632, China.

Digital Object Identifier 10.1109/TITS.2024.3432818

I. Introduction

With the rise of smart Internet of Things (IoT) devices (e.g., sensors, smartphones, cameras), there is a growing demand for computation-intensive tasks such as vehicle trajectory tracking, augmented reality, and face recognition. These tasks bring high energy consumption while the limited battery capacity and computing power of IoT devices adversely affect the user experience [1]. Edge Computing, an extension of Cloud Computing, enhances energy efficiency and reduces task processing latency for resource-limited IoT devices by positioning servers at the network edge [2].

However, the fixed locations of edge servers restrict their flexibility to adapt to the dynamic needs of mobile users, thereby limiting the capabilities of Edge Computing [3]. Owing to their low cost, high flexibility, and convenient deployment, UAVs serve as beneficial alternatives to static edge servers, especially in rural areas, maritime environments, and emergency situations (e.g. battlefield communication, earthquake rescue, and mountainous areas) [4]. By providing end-to-end communication with user devices, UAVs can function as "flying edge servers', offering considerable services to users within their communication coverage [5].

A. Current Advances

While UAVs aim to deliver flexible edge computing services, it poses challenges to determine the optimal deployment locations. Previous researches have extensively explored UAV deployment strategies to enhance network performance [6], [7], [8], [9], [10]. Xu et al. [11] proposed a constant factor approximation algorithm designed to minimize the number of UAVs needed for data collection. Lin et al. [12] offered a Line-of-Sight based method to refine UAV positioning for maximal coverage. Additionally, Han et al. [13] utilized a game theory approach to optimize UAV deployment for increased data collection and jamming resistance. However, an ideal UAV deployment strategy should not only be concerned with minimizing costs and maximizing system efficiency by using the fewest possible UAVs but also take into account the distribution of workload among them [14], [15], [16], [17]. The performance of a multi-UAV edge computing system is closely linked to how tasks are offloaded, as an edge server may suffer performance issues when overloaded with tasks [18], [19], [20], [21], [22]. Therefore, a well-designed joint optimization strategy for UAV deployment and task offloading is required

1558-0016 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

to optimize the energy consumption and delay performance of the UAV edge computing system.

Moreover, the energy consumption and delays in executing user tasks are significantly affected by the allocation of computation resources during the task offloading process [23], [24]. Zhang et al. [25] focused on joint optimization of computation resource allocation, UAV deployment, and offloading decisions through evolutionary algorithms. In addition, Reinforcement Learning (RL) has also been applied in UAV edge computing to address unknown network environments and the curse of dimensionality [26], [27], [28], [29]. Moreover, communication resources impact the data transmission rate [30], [31], [32], [33], [34]. These dependencies indicate that the efficiency of UAV edge computing systems hinges not only on deployment and offloading strategies but also on the effective management of computation and communication resources [35]. However, deployment and offloading strategies affect the resources that can be allocated. In turn, the allocated resource affects the quality of the deployment and offloading strategies, making them a type of closely coupled optimization variable [36], [37], [38]. Therefore, a comprehensive and energy-efficient UAV edge computing system necessitates a broader optimization scope, including computation and communication resources. As indicated in other works [39], [40], [41], such joint optimization is crucial for improving energy efficiency. However, the complexity of simultaneously optimizing multiple closely coupled variables significantly increases, which is a challenging issue. Furthermore, the collaboration among UAVs has been less considered, which is pivotal for a comprehensive and energy-efficient UAV edge computing system.

B. Motivation

Bridging the discussion from current advances to the emergent needs that drive our research, we now turn to clarify the motivation behind our proposed approach. In order to provide continuous computing services to all users, UAVs should be deployed to cover the locations of as many users as possible. Intuitively, optimizing UAV deployment can reduce economic costs and enhance system efficiency by deploying the minimum number of UAVs necessary to provide adequate services to all users. However, in real-world applications, user devices are typically unevenly distributed [42], which makes it challenging for UAVs to tradeoff where to deploy optimally and which users may need to be temporally excluded from service coverage. This uneven distribution means that UAVs covering areas with scattered users waste significant idle resources, while those covering densely populated areas are heavily burdened, leading to long queue time for tasks and low task execution efficiency. Such imbalanced load distribution among UAV edge servers diminishes system efficiency. Current research efforts to provide computing services to all users often result in deploying more UAVs, thereby increasing economic costs and the potential waste of idle resources.

Inspired by these resultant challenges, we develop a multi-UAV Collaborative edge Computing (UCC) system. As highlighted in Fig. 1, user tasks can be offloaded to UAVs and collaboratively processed among UAVs for computation.

C. Open Problems and Contributions

Currently, the full potential of UAV edge computing systems has not been fully realized. Therefore, in this paper we consider the collaborative computing among UAV edge servers to prevent network congestion or overload, thereby enhancing the performance of the UAV edge computing system. When considering the collaboration among UAVs, it is essential to jointly optimizing UAV deployment, collaborative task offloading, and the allocation of computation and communication resources. To be best of our knowledge, our work is the first one in providing a comprehensive approach to optimizing UAV deployment, task offloading, and resource allocation under uneven user distribution. The major contributions of this paper can be summarized as follows:

- We study the scenario of the UCC system that utilizes collaborative computing to split a computation task into some subtasks at a UAV and forward these subtasks to other UAVs. The objective of the joint optimization problem is to minimize the total energy consumption of the UCC system under the delay constraint, which is formulated as a mixed integer nonlinear programming (MINLP) problem. To simplify the overall complexity, the original MINLP problem is decoupled into a bi-level problem, and we propose a bi-level optimization framework called TDE to address it in a targeted manner.
- In the upper level, we propose an improved differential evolution (DE) algorithm to optimize the deployment of UAVs. We utilize the results of the greedy algorithm as the initial population in the initialization phase of DE to improve the iteration rate. In addition, we add the adaptive mutation factor and adaptive crossover factor to improve the iteration efficiency. Furthermore, we add the elimination operator to improve the quality of the optimization.
- In the lower level, the offloading decision, computation and communication resource allocation (i.e. transmission power) are optimized based on a Reinforcement Learning algorithm, called Twin Delayed Deep Deterministic policy gradient (TD3). The lower-level makes decisions by observing the UAV deployment provided by the upper-level and current network environment status of load.
- We conduct extensive experiments to evaluate the performance of the proposed TDE and the UCC system. Experimental results verify the effectiveness and superiority of multi-UAV collaborative computing, and demonstrate that our TDE outperforms the existing works (e.g., DDPG, ToDeTaS) and other benchmark schemes in terms of energy-efficiency and task completion rate under various network parameter settings.

The rest of the paper is organized as follows. We build the system model and formulate the target problem in Section II. Section III describes the details of our proposed UAV collaborative edge computing framework. Experimental results and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I
A QUALITATIVE COMPARISON OF RELATED WORKS WITH OURS

Work	UAV Deployment	Offloading Decision	Computation Resource allocation	Communication Resource allocation	Reinforcement Learning	Collaborative Computing
[6]	✓					
[14]	✓	✓				
[16]	✓	✓			✓	
[25]	✓	✓	✓			
[30]	✓	✓		✓		
[27]	✓			✓	✓	
[41]	✓		✓	✓		
Our work	✓	✓	✓	✓	✓	✓

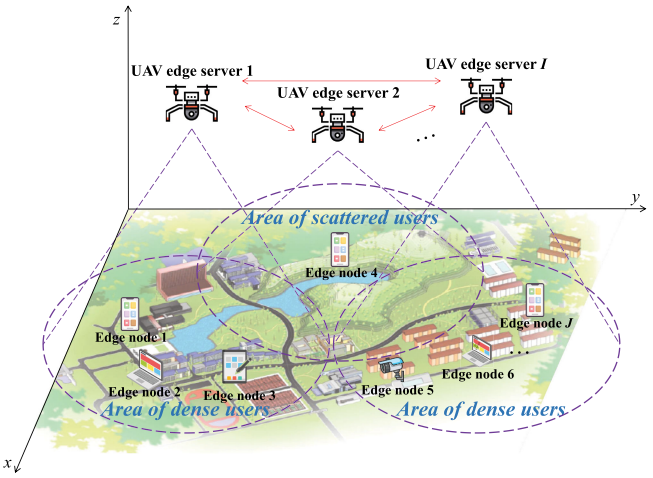


Fig. 1. The UCC system consists of multiple UAV edge nodes and multiple users.

TABLE II
IMPORTANT NOTATIONS

Notation	Description
$\mathcal{I}$	The set of UAV edge nodes
$\mathcal{J}$	The set of users
$\mathbf{L}$	The deployment of all UAVs
U_j	The task of user j
C_j	The computation workload of the task U_j
D_j	The size of input data for the task U_j
$T_j^{\max}$	The time delay constraint for task U_j
$\mathbf{O}$	The offloading decision matrix
$\Re$	The computation resource allocation matrix
$\mathbf{P}$	The power allocation matrix

analysis is provided to verify the performance of the TDE in Section IV. Section V analyzes potential challenges and future research directions. Finally, the conclusion is given in Section VI.

II. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first model the task local computation, the UAV offloading computation and the UAV collaborative computation. After that, the optimization problem is formulated to minimize energy consumption.

We consider the whole system including I UAVs with edge server integration (UAV edge nodes [1]) denoted as $\mathcal{I} = \{1, 2, \ldots, i, \ldots, I\}$ and J users denoted as $\mathcal{J} = \{1, 2, \ldots, j, \ldots J\}$. Fig. 1 depicts the proposed UCC system model. Without loss of generality, we construct a three-dimensional coordinate. The coordinate of user j is denoted as $(x_j, y_j, z_j), \forall j \in \mathcal{J}$. The coordinate of the UAV i is denoted as $l_i = (X_i, Y_i, H_i), \forall i \in \mathcal{I}$, where X_i and Y_i denote the horizontal coordinate and H_i is the altitude of UAV i. The deployment of all UAVs are denoted by $\mathbf{L} = \{l_1, l_2, \ldots, l_i, \ldots, l_I\}$. The task of user j is represented as $U_j = (D_j, C_j, T_j^{\max})$, where D_j denotes the size of input data, C_j is the computation workload, and $T_j^{\max}$ indicates the maximum time delay. All user task generations are modeled to follow a Poisson distribution. We ignore the delay caused by the transmission of the result due to the fact that the computation result is usually small compared to D_j. In addition, each UAV has two communication modules in the system. The first module has a directional antenna with a fixed beamwidth for user communication, and the second module has an omnidirectional antenna for UAV communication. Similar to the work in Table I, there is no infrastructure (e.g., Edge servers) in the proposed UCC system model. However, in contrast to a flying UAV edge server, a generic edge server has a fixed location. It can be intuitively seen that the relevant optimization problem becomes simpler by setting the coordinates to a fixed location. Therefore, the UCC system model under consideration and the proposed TDE framework are versatile and can be extended and adapted to scenarios with infrastructure by substituting a UAV with a fixed edge server in the model.

In the UCC system, the UAVs are networked in a fully interconnected manner, and they can offload redundant computation tasks to each other through collaborative computing to balance the load. Through collaborative computing, the UAVs can split a computation task into some subtasks and process them in parallel, note that splitting and forwarding each task at most once [43]. The collaborative offloading decision is generated based on the coordinate and load state of the UAV. After a task is offloaded to a UAV, the task can be performed independently in that UAV, or divided and forwarded to another UAV. The offloading decision for task U_j is represented as $\mathbf{o}_j = (\alpha_j, \beta_j^i, \mu_j^{i,k}), \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k$, where α_j denotes the local computation decision variable, β_j^i is the UAV offloading computation decision variable, and $\mu_j^{i,k}$ represents the UAV collaborative computation decision variable. Note that only one of the three variables can be 1 and the rest are 0.

- $\alpha_j = 1$ means that the user device processes the task U_j locally, otherwise, $\alpha_j = 0$;
- $\beta_j^i = 1$ indicates that the UAV i processes the task U_j independently, otherwise, $\beta_j^i = 0$.

[1] UAV edge node is called UAV hereafter for conciseness.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

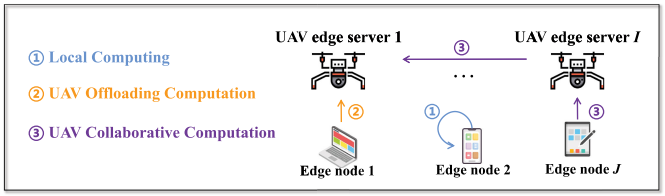


Fig. 2. Illustration of different task computation modes.

- $\mu_j^{i,k} = 1$ shows that the task U_j is divided to be computed collaboratively by UAV i and UAV k, since the computation resource of UAV i is only sufficient for computing part of the data and another UAV k has enough computation resource to compute the remaining data, otherwise $\mu_j^{i,k} = 0$.

The offloading decisions matrix for J users is denoted as $\mathbf{O}=\{\mathbf{o}_1,\ldots,\mathbf{o}_j,\ldots,\ \mathbf{o}_J\}$. The computation resource allocation matrix is denoted as $\Re$, and the computation resource allocated by UAV i to task U_j is represented as $r_j^i (j \in \mathcal{J}, i \in \mathcal{I})$. In particular, r_j^{loc} represents the local computation resource allocated by user device j to task U_j. $\mathbf{P} = \{p_j^i | p_j^i \in (0, P^{\max}], \forall j \in \mathcal{J}, \forall i \in \mathcal{I}\}$ denotes the power allocation matrix of user, where p_j^i is the transmission power when user j transmits U_j to UAV i.

In the following, we introduce the local computation model, UAV offloading computation model, and UAV collaborative computation model in detail.

A. Local Computation

As shown in Fig. 2, if $\alpha_j = 1$, the computing time of user j for the local computation task U_j is given by

$$T_{j,loc}^{comp} = \frac{D_j C_j}{r_j^{loc}}, \forall j \in \mathcal{J}. \tag{1}$$

The energy consumption for the local computation task U_j can be written as [18]:

$$E_{j,loc}^{comp} = k_0 \left(r_j^{loc}\right)^3 T_{j,loc}^{comp} = k_0 \left(r_j^{loc}\right)^2 D_j C_j, \forall j \in \mathcal{J}, \tag{2}$$

where $k_0 > 0$ is a constant parameter related to the CPU of the user device.

B. UAV Offloading Computation

In the case of task offloading, i.e. $\beta_j^i = 1$, the user must first transfer the computation tasks to a UAV, as shown in Fig. 2. The horizontal distance to UAV i for user j is given as

$$d_{j,i}^h = \sqrt{\left(x_j - X_i\right)^2 + \left(y_j - Y_i\right)^2}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{3}$$

When the task U_j is executed on UAV i, the user j should be within its coverage area. Thus, the following constraints need to be satisfied:

$$(\beta_j^i + \mu_j^{i,k}) d_{j,i}^h \le \mathrm{C}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}, \tag{4}$$

where C is the coverage radius of each UAV and $\mathrm{C} = H_i \tan\theta$.

The uplink data transmission rate when user j transmits the task U_j to UAV i can be expressed as

$$R_{j,i} = B\log_2\left(1 + \frac{p_j^i g_0 G_0}{B N_0 \theta^2 \left(\left(d_{j,i}^h\right)^2 + H_i^2\right)}\right), \forall j \in \mathcal{J}, \forall i \in \mathcal{I}, \tag{5}$$

where B is the bandwidth, p_j^i denotes the transmission power of user j, G_0 indicates a positive constant, g_0 represents the channel gain at the reference distance, and N_0 is the noise power spectral density.

The corresponding time to transmit the task U_j to UAV i can be expressed as

$$T_{j,i}^{trans} = \frac{D_j}{R_{j,i}}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{6}$$

The energy consumption to transmit the task U_j to UAV i can be expressed as

$$E_{j,i}^{trans} = p_j^i \frac{D_j}{R_{j,i}}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{7}$$

The computing time of U_j based on the computation resources r_j^i allocated by UAV i after the task being offloaded to the UAV can be written as

$$T_{j,i}^{comp} = \frac{D_j C_j}{r_j^i}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{8}$$

The energy consumption of UAV i for the computation task U_j can be expressed as

$$E_{j,i}^{comp} = k_1 \left(r_j^i\right)^2 D_j C_j, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}, \tag{9}$$

where $k_1 > 0$ is a constant parameter related to the CPU of the UAVs.

C. UAV Collaborative Computation

The distance between any two UAVs i and k is denoted as

$$d_{i,k}^{uav} = \sqrt{(X_i - X_k)^2 + (Y_i - Y_k)^2 + (H_i - H_k)^2}, \quad \forall i, k \in \mathcal{I}, i \ne k. \tag{10}$$

It is noted that to avoid collision, any two UAVs must maintain a minimum distance $d_{\min}^{uav}$. Therefore, the following constraints should be satisfied:

$$d_{i,k}^{uav} \ge d_{\min}^{uav}, \forall i, k \in \mathcal{I}, i \ne k. \tag{11}$$

The line-of-sight channel of the UAV communication link is dominant given the absence of obstacles in the air. Thus, the channel gain from UAV to UAV is described as the free-space path loss model. The link data transmission rate from UAV i to UAV k can be expressed as

$$R_{i,k}^{uav} = B\log_2\left(1 + \frac{p_i^{uav} g_0}{B N_0 \left(d_{i,k}^{uav}\right)^2}\right), \forall i, k \in \mathcal{I}, i \ne k, \tag{12}$$

where p_i^{uav} is the transmission power of UAV i.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

As shown in Fig. 2, when $\mu_j^{i,k} = 1$, UAV i computes part of the data D_j^i and forwards the remaining data D_j^r to UAV k that has enough computation resource for collaborative computing. UAV i only needs to compute part of the data D_j^i within the time delay constraint, and the subtasks do not affect each other's execution process after the task split. Thus, based on the given allocation of computation resources r_j^i, the size of the data D_j^i that can be computed within the time delay tolerance can be expressed as

$$D_j^i = \frac{\left(T_j^{\max} - T_{j,i}^{trans}\right) r_j^i}{C_j}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{13}$$

Accordingly, the size of the remaining data D_j^r to be forwarded can be expressed as

$$D_j^r = D_j - D_j^i, \forall j \in \mathcal{J}, \forall i \in \mathcal{I}. \tag{14}$$

The two UAVs will process the collaborative task in parallel when the UAV k with sufficient computation resource receives the remaining data D_j^r from UAV i. The time of transmitting the remaining data D_j^r from UAV i to UAV k can be expressed as

$$T_{i,k}^{trans} = \frac{D_j^r}{R_{i,k}^{uav}}, \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \tag{15}$$

The corresponding energy consumption of transmitting the remaining data can be expressed as

$$E_{i,k}^{trans} = p_i^{uav} \frac{D_j^r}{R_{i,k}^{uav}}, \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \tag{16}$$

In addition, the computation resources allocated to U_j by UAV k is denoted as r_j^k. Then, the computing time at UAV k is given by

$$T_{i,k}^{comp} = \frac{D_j^r C_j}{r_j^k}, \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \tag{17}$$

The energy consumption of the computation task U_j includes the energy consumption of UAV i and collaborative UAV k, which can be expressed as

$$E_{i,k}^{comp} = k_1 \left(r_j^i\right)^2 D_j^i C_j + k_1 \left(r_j^k\right)^2 D_j^r C_j, \quad \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \tag{18}$$

D. Problem Formulation

Firstly, the total time $\mathcal{T}_j^{sum}$ to transfer and compute task U_j can be expressed as

$$\begin{aligned} \mathcal{T}_j^{sum} = {} & \alpha_j T_{j,loc}^{comp} + \beta_j^i (T_{j,i}^{trans} + T_{j,i}^{comp}) \\ & + \mu_j^{i,k} (T_{j,i}^{trans} + T_{i,k}^{trans} + T_{i,k}^{comp}), \\ & \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \end{aligned} \tag{19}$$

Secondly, the total energy consumption $\mathcal{E}_j^{sum}$ to transfer and compute task U_j can be expressed as

$$\begin{aligned} \mathcal{E}_j^{sum} = {} & \alpha_j E_{j,loc}^{comp} + \beta_j^i (E_{j,i}^{trans} + E_{j,i}^{comp}) \\ & + \mu_j^{i,k} (E_{j,i}^{trans} + E_{i,k}^{trans} + E_{j,i}^{comp} + E_{i,k}^{comp}), \\ & \forall j \in \mathcal{J}, \forall i, k \in \mathcal{I}, i \neq k. \end{aligned} \tag{20}$$

The total energy consumption of the considered UCC system is jointly determined by the UAV deployment, task collaborative offloading decision, computation and communication resource allocation. The joint optimization problem can be formulated as

$$\mathbf{P1}: \min_{\mathbf{L},\mathbf{O},\Re,\mathbf{P}} \mathbb{E}^{sum} = \sum_{j=1}^{J} \mathcal{E}_j^{sum} + \sum_{i=1}^{I} (E_i^h + E_i^f) \tag{21}$$

Subject to

$$d_{i,k}^{uav} \geq d_{\min}^{uav}, \forall i, k \in \mathcal{I}, i \neq k \tag{22}$$

$$(\beta_j^i + \mu_j^{i,k}) d_{j,i}^h \leq \mathrm{C}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I} \tag{23}$$

$$H_{max} \leq H_i \leq H_{min}, \forall i \in \mathcal{I} \tag{24}$$

$$\alpha_j, \beta_j^i, \mu_j^{i,k} \in \{0, 1\}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I} \tag{25}$$

$$\sum_{i \in \mathcal{I}} \sum_{k \in \mathcal{I} - \{i\}} (\alpha_j + \beta_j^i + \mu_j^{i,k}) \leq 1, \forall j \in \mathcal{J} \tag{26}$$

$$\sum_{j=1}^{J} r_j^i \leq r_{\max}^i, \forall i \in \mathcal{I} \tag{27}$$

$$r_j^{loc} \leq r_{\max}^j, \forall j \in \mathcal{J} \tag{28}$$

$$p_j^i \leq P_{\max}, \forall j \in \mathcal{J}, \forall i \in \mathcal{I} \tag{29}$$

$$0 < \mathcal{T}_j^{sum} \leq T_j^{\max}, \forall j \in \mathcal{J} \tag{30}$$

where constraint (22) guarantees that two UAVs must maintain a minimum distance $d_{\min}^{uav}$ to avoid collision; constraint (23) indicates that user j must be within the coverage area of UAV i if the task U_j is offloaded to UAV i; constraint (24) limits UAV flight to altitudes between the minimum height H_{min} and the maximum height H_{max}. constraint (25) corresponds to three offloading decision variables; constraint (26) guarantees that only one offloading decision can be selected for a task; constraint (27) denotes the allocated maximum computation resource of UAV i; constraint (28) indicates the maximum computation resources of user device; constraint (29) denotes the maximum transmission power of the user; constraint (30) denotes the delay tolerance of each task; and $E_i^h = p^{hover} T_j^{\max}$ is the hovering energy consumption of UAV and p^{hover} denotes the hovering power [24]; and $E_i^f = p^{fly}(d_i^{fly}/V^{fly})$ is the flying energy consumption of UAV, where p^{fly} denotes flying power, d_i^{fly} indicates the UAV flying distance after adjusting deployment locations, V^{fly} corresponds to the flying speed of UAV [44].

III. UAV Collaborative Edge Computing Framework

In this section, we first analyze the problem and transform the original MINLP problem into a bi-level optimization problem, and then we present the details of the proposed bi-level optimization framework TDE.

A. Transformation Into Bi-Level Problem

Through the analysis of **P1**, we can observe that problem **P1** is a mixed-integer nonlinear programming (MINLP) problem. It is generally a challenge to solve the problem **P1** directly with the traditional optimization method. Moreover, we can

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

find that problem **P1** has the following two characteristics. First, the performance of the UAV deployment cannot be evaluated before offloading decision and resource allocation are generated. Secondly, according to the result of the UAV deployment, the feasible task offloading decisions and the number of available resources may vary, which requires that both should be considered in advance.

It is known that the bi-level optimization method can solve the problem with mutually constraining features [18]. The upper-level problem is solved under the condition of ensuring the optimization of the lower-level problem as a prerequisite. Thus we transform **P1** into a bi-level optimization problem, i.e., the UAV deployment strategy will be achieved by solving the upper-level optimization problem **P2**, and the task offloading decision, computation resource, and communication resource allocation strategy will be given by solving the lower-level optimization problem **P3**. The optimization objective is to maximize the energy-efficiency of the UCC system.

In the upper level, optimizing UAV deployment is a typical NP-hard problem [42], making proposing an optimal UAV deployment strategy be a challenging task. As the network's scale expands, the computational complexity increases significantly, rendering it impractical to derive an optimal solution through deterministic algorithms within a reasonable time frame. Differential Evolution (DE) stands out as one of the most efficient evolutionary algorithms for addressing diverse optimization problems, drawing inspiration from biological evolution. This approach involves retaining a population that adapts to the environment through repeated iterations. This DE algorithm has high efficiency in handling complex optimization problems, particularly in continuously variable spaces, thus it perfectly suits the requirements for dynamic UAV deployment, which is the reason why we choose the DE algorithm to optimize the placement of UAVs.

Notably, by encoding the coordinates of the UAVs (as elaborated in the next subsection), the optimization variables for UAV deployment will be continuous, in which DE outperforms other evolutionary algorithms in continuous optimization [45]. Thus, we propose an optimization algorithm for UAV deployment based on DE, modifying the original DE algorithm to suit the particulars of the problem **P2**. The objective of the upper-level is to optimize the UAV deployment based on the $\mathbf{O}^*$, $\Re^*$, and $\mathbf{P}^*$ provided by the lower-level. We formulate the upper-level optimization problem **P2** as

$$\mathbf{P2}: \min_{\mathbf{L}} \mathbb{E}^{sum} = \sum_{j=1}^{J} \mathcal{E}_j^{sum}(\mathbf{O}^*, \Re^*, \mathbf{P}^*) + \sum_{i=1}^{I}(E_i^h + E_i^f)$$
$$\textbf{Subject to} \quad (22), (23), (24). \tag{31}$$

In the lower level, according to the UAV deployment strategy in the upper level, we can obtain the task offloading decision, computation and communication resource allocation. The goal is to minimize the total energy consumption of the system. The deployment of UAVs and the network load status change over time, making it difficult for traditional methods to guarantee optimal solutions. Deep reinforcement learning (DRL), on the other hand, has advantages in handling high-dimensional action spaces and adapting to the constantly changing dynamics of the environment, which can satisfy the unpredictable nature of task demands and the need for efficient resource management in UAV-edge computing systems. In DRL, agents construct optimal strategies by interacting with the unknown environment through repetitive observations, behaviors and rewards.

We propose an optimization algorithm in the lower-level based on DRL method, i.e., Twin Delayed Deep Deterministic policy gradient (TD3) algorithm, for the offloading decision and the allocation of resources in the case that the changing UAV deployment and network load status affect the feasible task offloading decisions and the number of available resources. In the lower level, we obtain the optimal offloading decisions and resource allocation based on the deployment of UAVs in the upper-level. We formulate the lower-level optimization problem **P3** as

$$\mathbf{P3}: \min_{\mathbf{O},\Re,\mathbf{P}} \mathbb{E}^{sum} = \sum_{j=1}^{J} \mathcal{E}_j^{sum}(\mathbf{L}^*) + \sum_{i=1}^{I}(E_i^h + E_i^f)$$
$$\textbf{Subject to} \quad (23), (25), (26), (27), (28), (29), (30). \tag{32}$$

In **P2** and **P3** the dependence of the offloading decision and the resource allocation on the UAV deployment can be fully considered. Specifically, since the offloading decision and the resource allocation are generated based on the UAV deployment, the limitations on feasible task offloading decisions and the number of available resources caused by the UAV deployment are considered. In addition, the quality of UAV deployment and the energy consumption of the overall system can only be accurately evaluated when the optimal offloading decisions and the resource are obtained at the lower-level. Specifically, the upper-level optimization algorithm is nested within the TD3 reinforcement learning framework to constructed the bi-level optimization framework TDE.

B. Upper-Level Optimization

DE algorithm is a well-known optimization algorithm based on the biological evolution process. We construct a UAV Deployment Generation (UDG) algorithm based on a modified DE algorithm. The UDG algorithm optimizes the deployment of UAVs according to the offloading decision and resource allocation derived by the lower-level optimization. Compared to the benchmark DE algorithm that generates populations randomly in the initialization phase, we utilize the result of the greedy algorithm as the initial population to accelerate the iteration rate; in addition, we add the adaptive mutation factor and adaptive crossover factor to improve the iteration efficiency of the algorithm; furthermore, we add the elimination operator to improve the quality of optimization. UDG executes global search well, which quickly searches the optimal solution in the solution space without being trapped in the local optimal solution.

Specifically, we first encode the coordinate of each UAV and give the fitness function to evaluate the optimization results; and then we propose a greedy algorithm based population initialization; in the end, we propose the mutation, crossover, elimination and selection processes in the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 UDG Algorithm

Input {**O**,**W**,**P**} from the lower-level, reevaluate the system energy consumption of the entire population Pop;
for $g = 1$ to G **do**
  Update the adaptive mutation factor $\mathcal{F}$ based on (36);
  Perform the mutation operator based on (35);
  Update the adaptive crossover factor Cr based on (39);
  Perform the crossover operator based on (38);
  Perform the elimination operator and calculate the fitness based on (34);
  for $n = 1$ to $\mathcal{P}_{num}$ **do**
    Implement the selection operator based on (40);
  end for
end for
A new population is generated, and the optimal UAV deployment **L** is selected;
return L

Algorithm 2 Greedy Initialization

Initialize adaptive mutation factor $\mathcal{F}$, adaptive crossover factor C_r, population size $\mathcal{P}_{num}$;
Initialize {**O**,**W**,**P**} with random initial values;
Initialize population $Pop = \{\mathbf{L}_1, \ldots, \mathbf{L}_n, \ldots, \mathbf{L}_{\mathcal{P}_{num}}\}$;
for $n = 1$ to $\mathcal{P}_{num}$ **do**
  Randomly generate J UAV individuals around all users;
  repeat
    Select the UAV individual that covers most of the users and put it into $\mathbf{L}_n$;
    Remove the users that have been covered by the remaining UAVs;
  until No UAV individual can cover the user or any constraint among (22)-(30) is not satisfied anymore
end for
Evaluate the system energy consumption of the entire population Pop according to the random {**O**,**W**,**P**} and select the optimal initialization UAV deployment **L**;
return L

evolutionary algorithm. The UDG algorithm is presented in **Algorithm** 1. The specific steps are discussed below.

*1) **Encoding**:* The number of UAVs may change from one generation to another generation due to the optimization in the evolutionary process. In typical DE, the length of each individual in the generation is variable. In this case, the deployment of UAVs is a variable-length optimization problem, it is still challenging for typical DE to address variable-length optimization problems [46]. Thus, an encoding mechanism is proposed in which the coordinate of each UAV is encoded as an individual, which is given by

$$\xi_i = (X_i, Y_i, H_i), i \in \mathcal{I}, \tag{33}$$

and the entire population represents the entire deployment. This encoding mechanism has two advantages. First, rather than a variable length, each individual has a fixed length during the evolution; second, we can optimize the UAV deployment in a low-dimensional search space.

*2) **Fitness Function**:* The objective of the upper-level optimization is to optimize the UAV deployment given the offloading decision and resource allocation from the lower-level to minimize the total energy consumption of the UCC systhem. Therefore, the fitness function in UDG can be expressed as

$$F = -\mathbb{E}^{sum}. \tag{34}$$

In this case, less energy consumption corresponds to higher fitness value, indicating easier retainment of future generations.

*3) **Population Initialization**:* The traditional DE simply generates an initial population with random, which leads to slow and inefficient iterations. Therefore, we generate an initial optimized population based on a greedy algorithm to improve the iteration efficiency. Specifically, we first initialize {**O**,**W**,**P**} with random, and then initialize population $Pop = \{\mathbf{L}_1, \ldots, \mathbf{L}_n, \ldots, \mathbf{L}_{\mathcal{P}_{num}}\}$. Note that the size of populations is $\mathcal{P}_{num}$. We generate J UAV individuals in such a way that there is one UAV around each user. Then, we select the UAV individual that covers the most users and put it into $\mathbf{L}_n$, and remove the already covered users from the remaining UAVs. Repeat the process until no UAV individual can cover the user or constraint (22)-(30) cannot be satisfied. Thus, we generate the population Pop. The greedy initialization is presented in **Algorithm** 2.

*4) **Mutation**:* The UDG realizes mutation operator through differential method enabling local random search and avoiding premature convergence. Therefore, three different UAV individuals $\xi_1(g)$, $\xi_2(g)$, $\xi_3(g)$ can be randomly selected in the population, and the produced mutation individual can be expressed as:

$$I_n(g) = \xi_1(g) + \mathcal{F}(\xi_2(g) - \xi_3(g)), \tag{35}$$

where $\mathcal{F}$ is the adaptive mutation factor, which is defined as follows:

$$\mathcal{F} = \mathcal{F}_0 \cdot 2^{\lambda}, \tag{36}$$

where $\mathcal{F}_0$ is the initial mutation factor, and λ is a value that varies according to the number of iterative rounds, which is defined by

$$\lambda = e^{1-\frac{g}{g+1-G}}. \tag{37}$$

In the initial iteration of the algorithm, the mutation factor has a large value to ensure the diversity of individuals and avoid premature maturity. As iteration of the algorithm grows, both the mutation factor and the mutation rate gradually decrease, which preserves the superior information to avoid the destruction of the optimal solution. Therefore, the adaptive mutation factor can increase the probability of searching for the global optimal solution.

*5) **Crossover**:* The UDG uses crossover operator to maintain the diversity of the population, and crossover operator will be performed with the n-th individual $\xi_n(g)$ in the population

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

and mutated intermediate $I_n(g)$ to produce the experimental individual $H_n(g)$. The crossover operator is given by

$$H_n(g) = \begin{cases} I_n(g) & \text{rand}(0,1) \le Cr \\ \xi_n(g) & \text{otherwise,} \end{cases} \tag{38}$$

where Cr is the adaptive crossover factor, which is defined by

$$Cr = (Cr_{\max} - Cr_{\min})(\frac{g}{G})^2 + Cr_{\min}, \tag{39}$$

where $Cr_{\max}$ is the maximum crossing probability, $Cr_{\min}$ is the minimum crossing probability, and G represents the maximum number of iterations. In the initial iteration of the algorithm, the crossover factor has a small value to ensure the diversity of individuals and global search. As the algorithm iterates, the value of the crossover factor gradually increases to accelerate convergence and local search. Therefore, the adaptive mutation factor is to slow down the increasing speed of the cross factor and strengthen the global optimization ability of the algorithm.

*6) **Elimination**:* The UDG implements elimination operator on all experimental individuals in the population. Specifically, the UAV experimental individual $H_n(g)$ that covers the least users in each population will be temporarily removed, and the fitness will be calculated based on (34) afterwards. The elimination operator succeeds with improved fitness, otherwise the elimination operator will be reversed.

*7) **Selection**:* The UDG applies a "greedy" selection strategy. After the mutation, crossover, and elimination operators being excecuted, the experimental individual $H_n(g)$ is generated to compete with the individual $\xi_n(g)$ in the population. Only if $H_n(g)$ has a better fitness $F(H_n(g))$ than $F(\xi_n(g))$, $H_n(g)$ will be selected as the filial generation; otherwise, $\xi_n(g)$ will be directly selected as the filial generation. The selection operator is given by

$$\xi_n(g+1) = \begin{cases} H_n(g) & \text{if} F(H_n(g)) \ge F(\xi_n(g)) \\ \xi_n(g) & \text{otherwise,} \end{cases} \tag{40}$$

The above steps are repeated until the optimized UAV deployment is obtained or the maximum number of iterations is reached. Note that all UDG processes are performed under the constraints (22)-(30).

C. Lower-Level Optimization

The offloading decision and the resource allocation algorithm based on TD3 is proposed according to the deployment of UAVs provided by the upper-level. The offloading decision and the resource allocation is optimized by the lower-level and then feeds back to the upper-level. In this part, we first formulate the decision problem as an Markov Decision Process (MDP), and design a TD3-based algorithm, in order to find the best policy that governs the optimal offloading decision and resource allocation for adapting the UAV deployment provided by the upper-level within unknown environment.

*1) **MDP-Based DRL Framework**:* In this part, MDP is usually represented by 4-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, in which

- The state space $\mathcal{S}$ contains all possible states s_x in the environment, where $s_x \in \mathcal{S}$.
- The action space $\mathcal{A}$ contains all feasible actions a_x, where $a_x \in \mathcal{A}$.
- A transition distribution $\mathcal{P}$, where $P(s_{x+1}|s_x, a_x), s_x, s_{x+1} \in \mathcal{S}, a_x \in \mathcal{A}$ is the probability that leads to the new state, s_{x+1}, after executing an action a_x at the state s_x.
- The state space $\mathcal{S}$ contains all possible states s_x in the environment, where $s_x \in \mathcal{S}$.

To learn reward-maximizing behavior, DRL considers the paradigm of agent-environment interaction. The objective of RL is to find the optimal policy π^*. The agent chooses actions $a_x \in \mathcal{A}$ according to its policy $\pi : s_x \to a_x$ in a given state $s_x \in \mathcal{S}$ and obtains a reward $\mathcal{R}$ and the new state s_{x+1} of the environment. Under the MDP framework, we will elaborate the state space, action space and system reward sequentially as follows:

- **State Space** ($\mathcal{S}$): $\mathcal{S} \triangleq [s_1, s_2, \ldots, s_x, \ldots, s_n]$ the indicates the state space, where s_x indicates current system state of UAV deployment and load, i.e. the coordinates and remaining available resources of UAVs. After performing the action and receiving feedback from the upper-level, the current system environment, i.e. the coordinates and resource residuals in the new state s_{x+1} will be updated.
- **Action Space** ($\mathcal{A}$): $\mathcal{A} \triangleq \{a_1, a_2, \ldots, a_x, \ldots, a_n\}$ indicates the action space. Action a consists of three parts: collaborative offloading decision, power allocation, and computation resource allocation. Vector $\mathbb{O} = [\mathbf{o}_1, \ldots, \mathbf{o}_j, \ldots, \mathbf{o}_J]$ indicates the collaborative offloading decision, where $\mathbf{o}_j = \{\alpha_j, \beta_j^i, \mu_j^{i,k}\}$ denotes the collaborative offloading decision for user j. The power selection is a vector $\Pi = [p_1, \ldots, p_j, \ldots, p_J]$, where p_j represents the transmission power configuration for user j, constrained by the maximum transmission power $P_{\max}$. For the computation resource allocation $\Re$, its processes are given as follows: By analyzing the computation models (2), (9), (18), it is observed that a strict monotonic increasing relationship exists between computation resources and the total computation energy consumption. For the purpose of minimizing the total computation energy consumption, the computation resources allocated to each task should not exceed what it needs. Thus, we can obtain the optimized computation resource allocation $\Re$ directly through monotonic optimization method according to the lower bound of computation resource. The lower bound of computation resource for task U_j can be derived according to the computation models (2), (9), (18) and the delay tolerance (19) based on the maximum delay constraint $T_j^{\max}$. The lower bound can be expressed as

$$\begin{cases} r_j^{loc} = \dfrac{D_j C_j}{T_j^{\max}}, & \text{if } \alpha_j = 1 \\ r_j^{i} = \dfrac{D_j C_j}{T_j^{\max} - T_{j,i}^{trans}}, & \text{if } \beta_j^i = 1 \\ r_j^{k} = \dfrac{D_j^r C_j}{T_j^{\max} - T_{j,i}^{trans} - T_{i,k}^{trans}}, & \text{if } \mu_j^{i,k} = 1. \end{cases}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

$$\forall j \in \mathcal{J}, i, k \in \mathcal{I}, i \neq k. \tag{41}$$

We can observe from these lower bound equations that once the UAV deployment $\mathbf{L}$, offloading decision $\mathbf{O}$, and power allocation $\mathbf{P}$ are given, the lower bound of the computation resource for task U_j can be determined. Therefore, by combining vectors $\mathbb{O}$, Π, and $\Re$, the system action becomes

$$a = \begin{bmatrix} \mathbf{o}_1, \ldots, \mathbf{o}_j, \ldots, \mathbf{o}_J \\ p_1, \ldots, p_j, \ldots, p_J \\ r_1, \ldots, r_j, \ldots, r_J \end{bmatrix}.$$

Accordingly, we can solve the lower-level action selection problem to derive the collaborative offloading decision $\mathbf{O}^*$, power allocation $\mathbf{P}^*$, and the computation resource allocation $\Re^*$.

- **System Reward** ($\mathcal{R}$): After an action a is selected in a state s, a reward $\mathcal{R}$ should be derived. The optimization variables $\{\mathbf{L}, \mathbf{O}, \Re, \mathbf{P}\}$ can be derived after the UAV deployment is obtained by the upper-level and the action is determined in the lower-level. Thus, we can derive the total energy consumption $\mathbb{E}^{sum}$ according to the objective function (21), which can be used as the system reward. RL aims to maximize the total reward, therefore we denote the system reward by $-\mathbb{E}^{sum}$. The minimum energy consumption corresponds to the maximum reward.

Note that the next UAV deployment state information can be observed from the upper-level feedback and the remaining resources can be calculated by (41), which can be easily verified as Markovian. Generally, an MDP problem can be solved using Dynamic Programming algorithms or Linear Programming. However, since the UAV is deployed with no prior knowledge on the environment, then Dynamic Programming or Linear Programming algorithms cannot be leveraged to solve the formulated MDP problem. Thus, to solve the formulated MDP problem in absence of the state transition probabilities $\mathcal{P}$ of the Markov model, the DRL algorithm TD3 is employed in the next subsection.

*2) **TD3-Based Algorithm**:* We utilize the Twin Delayed Deep Deterministic policy gradient (TD3) method to solve the offloading decision and resource allocation problem. TD3 addresses the overestimation bias that is common in other DRL algorithms, enhancing the stability of learning. This is particularly important for problem P3, where stability is crucial for consistent energy management over time. It is well known that conventional DDPG is generally not robust enough to adjust hyperparameters and other fine adjustments. Thus, TD3 makes improvements based on the DDPG by introducing the following techniques [47]. First, to relieve the overestimation problem in DDPG, the clipped double-Q-learning technique is utilized to build the objective function by learning two Q-functions and using the Q-function with a smaller Q-value. In addition, to enhance a more stable training policy, the delayed policy updates technique is implemented to make the update frequency of the actor network lower than the critic network. Furthermore, to achieve stability of policy fluctuation and make the policy hard to exploit the errors in the Q-functions, the target policy smoothing technique is utilized by inserting random noise to the target action and smooth the changes in the Q-function along with different actions. The specifics processes of TD3-based algorithm are depicted below.

Algorithm 3 TDE

1: Initialize critic networks and actor network with random parameters ϑ_1, ϑ_2, ϕ;
2: Initialize target networks $\vartheta_1' \leftarrow \vartheta_1$, $\vartheta_2' \leftarrow \vartheta_2$, $\phi' \leftarrow \phi$;
3: Initialize replay memory $\mathcal{V}$;
4: Initialize a UAV deployment population Pop based on **Algorithm 2**;
5: **for** $iter = 1, 2, 3, \ldots, Z$ **do**
6: Observe current environment state s_x, i.e., the deployment and load of UAVs;
7: Select action a_x with exploration noise based on (43);
8: Obtain UAV deployment $\mathbf{L}$ based on **Algorithm 1** for a given $a_x = \{\mathbf{O}, \mathbf{W}, \mathbf{P}\}$;
9: Perform action a^x and obtain reward $\mathcal{R}_x$ and state s_{x+1};
10: Store transition tuple $(s_x, a_x, \mathcal{R}_x, s_{x+1})$ in $\mathcal{V}$;
11: Randomly sample mini-batch of $\mathcal{X}$ transitions $(s_y, a_y, \mathcal{R}_y, s_{y+1})$ from $\mathcal{V}$;
12: Added noise to action a_y based on (43);
13: Update critic network parameters ϑ_1,ϑ_2 based on (44);
14: **if** $t \ mod \ \delta = 0$ **then**
15: Update the actor network parameters ϕ by policy gradient based on (45);
16: Update target network parameter ϑ_1', ϑ_2' and ϕ' based on (46) and (47);
17: **end if**
18: **end for**

Similar to the conventional Deep Deterministic Policy Gradient (DDPG) [48], TD3 is based on the actor-critic structure. Specifically, the learning scheme is comprised of six neural networks: one actor network, one actor target network with parameters ϕ and ϕ', two critic networks and the corresponding target critic networks with parameters ϑ_1, ϑ_2 and ϑ_1', ϑ_2'. The actor network is dedicated to the selection of a possible action with reference to the relevant state, and the critic network is committed to evaluating the performance of this action. The objective of the target network is to be responsible for stabilizing the entire learning process.

It is well known that conventional DDPG is generally not robust enough to adjust hyperparameters and other fine adjustments. Thus, TD3 makes improvements based on the DDPG by introducing the following techniques [47]. First, to relieve the overestimation problem in DDPG, the clipped double-Q-learning technique is utilized to build the objective function by learning two Q-functions and using the Q-function with a smaller Q-value. In addition, to enhance a more stable training policy, the delayed policy updates technique is implemented to make the update frequency of the actor network lower than the critic network. Furthermore, to achieve stability of policy fluctuation and make the policy hard to exploit the errors in the Q-functions, the target policy smoothing technique is utilized by inserting random noise to the target action and smooth

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

the changes in the Q-function along with different actions. The specifics processes of TD3-based algorithm are depicted below.

3) Critic Network Update: With clipped double-Q-learning, the smaller Q-value of the two critic target networks is selected as the target value y_x when the critic network updates its parameters. The target value y_x can be expressed as

$$y_x = \mathcal{R}_x + \gamma \min_{n=1,2} Q_{\vartheta'_n}(s_{x+1}, \widetilde{a}_x), \tag{42}$$

where γ is the discount factor, and $\widetilde{a}_x$ denotes the target action after adding random noise by utilizing target policy smoothing technique to achieve stability of policy fluctuation and the algorithm. The target action $\widetilde{a}_x$ can be expressed as

$$\widetilde{a}_x = \pi_{\phi'}(s_{x+1}) + clip(\eta, -c, c), \tag{43}$$

where $\eta \in N(0, \sigma)$ and c are the noise clip.

Then, the critic network is trained to update the parameters ϑ_1 and ϑ_2 by

$$\vartheta_n \leftarrow \arg\min_{\vartheta_n} \frac{1}{\mathcal{X}} \sum_{x=1}^{\mathcal{X}} (y_x - Q_{\vartheta_n}(s_x, a_x))^2, n = 1, 2, \tag{44}$$

where $\mathcal{X}$ is the batch size.

4) Actor Network Update: The actor network optimizes the objective function by the deterministic policy gradient method. The parameter update of the actor network can be expressed as

$$\nabla_\phi J(\phi) = \frac{1}{\mathcal{X}} \sum_{x}^{\mathcal{X}} \nabla_a Q_{\vartheta_1}(s_x, a_x)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s_x) \tag{45}$$

5) Target Network Update: To avoid the problem of targets moving during the learning process, the parameter update of the target networks implements a soft update mechanism to stabilize learning and enhance performance, which be expressed as follows:

$$\vartheta'_n \leftarrow \tau\vartheta_n + (1-\tau)\vartheta'_n, n = 1, 2, \tag{46}$$

$$\phi' \leftarrow \tau\phi + (1-\tau)\phi', \tag{47}$$

where τ denotes the learning rate. Note that with the modification of delayed policy updates, we only update the actor and target networks after a fixed number of updates δ to the critic.

In summary, the proposed bi-level optimization framework TDE is presented in **Algorithm 3**. First, the relevant parameters such as actor network parameters and critic network parameters are initialized (Line 1-3). According to the upper-level, an initial UAV deployment population is generated based on the greedy initialization algorithm (Line 4). In the TDE iterative process, an action is selected based on the currently observed environment state, which is the offloading decision and resource allocation, and then it is fed back to the upper-level to obtain the UAV deployment (Line 6-10). For the training process (Line 11-18), mini-batches of $\mathcal{X}$ transitions are randomly sampled from the relay memory $\mathcal{V}$. Update the critic network by minimizing the loss function (Line 12-14). Furthermore, delayed policy updates consist of updating the actor network and all target networks every δ iterations (Line 15-18).

TABLE III
EXPERIMENTAL PARAMETER SETTINGS

Parameter	Value	Parameter	Value
B	1 MHz	$r^i_{\max}$	10 GHz
θ	$\frac{\pi}{4}$	$r^j_{\max}$	[0, 0.9] GHz
g_0	1.42×10^{-4}	$\mathcal{F}_0$	0.3
G_0	2.2846	$Cr_{\max}$	0.9
N_0	10^{-20} W/Hz	$Cr_{\min}$	0.3
p_i^{uav}	1 W	$\mathcal{P}_{num}$	50
p^{hover}	1000 W	G	50
$P_{\max}$	1 W	Z	300
$T_j^{\max}$	1 s	σ	0.2
D_j	[10, 50] KB	c	1
C_j	1000 cycles/bit	$\mathcal{X}$	64
$d^{uav}_{\min}$	10 m	τ	3×10^{-4}
γ	0.9	δ	3
k_0	10^{-25}	k_1	10^{-26}
p^{fly}	4W	V^{fly}	5 km/h
H_{min}	50 m	H_{max}	100 m

TABLE IV
SIDE LENGTHS OF SQUARE AREA WITH DIFFERENT NUMBER OF USERS

N	100	200	300	400	500
Side Length (m)	320	450	550	640	710
N	600	700	800	900	1000
Side Length (m)	780	840	900	950	1000

Next, a complexity analysis for the main part of **Algorithm 3** is provided. In the initialization stage in Line 4, **Algorithm 2** is called to generate the initial UAV deployment. There may be at most J greedy selections for each population. Therefore, its computational complexity is $\mathcal{O}(\mathcal{P}_{num} J)$. For each iteration of the main part of TDE in Line 8, the **Algorithm 1** is called to evaluate UAV deployment $\mathcal{P}_{num} \times G$ times. Thus, its computational complexity is $\mathcal{O}(\mathcal{P}_{num} GZ)$. Overall, the computational complexity of TDE is $\mathcal{O}(\mathcal{P}_{num}(J + GZ))$.

Once the neural network is trained, the UAV deployment can be obtained at the upper-level, and the collaborative offloading decision, power and computation resource allocation based on the action selection can be obtained at the lower-level. Thus, the optimal solution and the corresponding minimum energy consumption $\mathbb{E}^*_{sum}$ can be obtained.

IV. EXPERIMENTAL STUDY

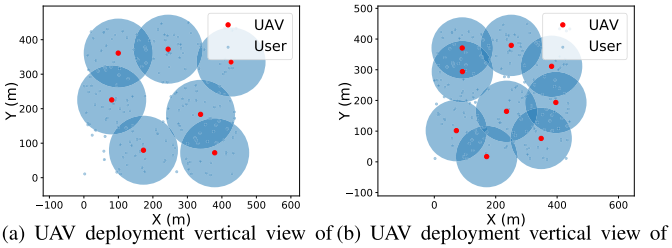
A. Experimental Settings

The experimental parameter settings of the proposed TDE are given in Table III [24], [35]. In addition, both the actor and critic networks consist of 2 hidden layers, where the related number of neurons is 32, note that the corresponding target network has the same network architecture. Moreover, all users are located on a plane with zero altitude. Ten instances with different numbers of users are provided to study the performance of TDE: J = 100, 200, ..., 1000. As shown in Table IV, we assume that all users are distributed in square areas with different side lengths.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF SCHEMES UNDER UNIFORM DISTRIBUTION OF USERS IN TERMS OF AVERAGE EC (JOULE) AND AVERAGE NC

Uniform J	ToDeTaS		DDPG		No Collaboration		TDE	
	Average EC	Average NC	Average EC	Average NC	Average EC	Average NC	Average EC	Average NC
100	5138.22	100	4739.17	**93**	**5204.40**	100	**4204.16**	**100**
200	8690.85	200	8178.26	176	9524.65	200	**7511.35**	**200**
300	15793.75	300	12375.94	280	15756.69	300	**11811.99**	**300**
400	22248.95	395	18818.65	365	21112.10	397	**16178.09**	**400**
500	29639.29	490	25547.30	458	24212.40	485	**18483.56**	**500**
600	33949.44	588	27689.97	550	31441.08	578	**23567.82**	**600**
700	39696.53	687	34534.21	635	35714.85	667	**26860.26**	**700**
800	43061.69	786	38409.30	711	41940.66	774	**30371.81**	**800**
900	48447.18	875	43275.50	828	45209.32	859	**32176.14**	**900**
1000	**52050.34**	970	49303.65	**916**	49505.83	957	**35174.91**	**1000**



(a) UAV deployment vertical view of TDE (b) UAV deployment vertical view of ToDeTaS

Fig. 3. Comparison of UAV deployment at $J = 200$ uniform distribution.

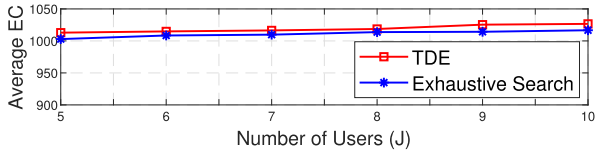


Fig. 4. Comparison with exhaustive search.

We adopt two performance metrics to evaluate the effectiveness of the proposed TDE, i.e., Number of Completed tasks (NC) and total Energy Consumption (EC) in the unit of Joule. The NC denotes the completion rate of tasks within the delay tolerance, and the EC denotes the energy consumption of the users in the UCC system.

B. Effectiveness of TDE

Firstly, we compare our method with Exhaustive Search (ES). ES is a brute force method that finds the global optimal solution for optimization problems by enumerating and comparing all feasible solutions. The EC performance of TDE compared to ES on a small user scale is illustrated in Fig. 4.

It can be observed that the performance of the proposed TDE closely matches the global optimum obtained by ES and the gap between the near-optimal and the optimal solution is no more than 1.1%. Therefore, the proposed algorithm is a practical and efficient solution.

Moreover, in order to evaluate the effectiveness of TDE, we compare the proposed scheme with the following three schemes:

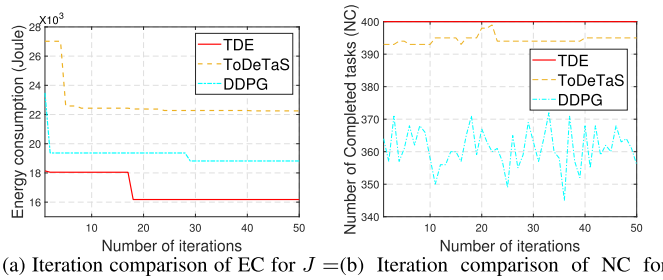
- ToDeTaS: ToDeTaS is a bi-level algorithm for UAV deployment optimization based on the benchmark differential evolution algorithm proposed by Wang et al. [24]. ToDeTaS utilizes DE to obtain UAV deployment in the upper-level and greedy algorithm to obtain task scheduling in the lower-level. Note that we employ collaborative computing for ToDeTaS in order to conduct a fair comparison experiment.
- DDPG: DDPG obtains the collaborative offloading decision and resource allocation through action selection in the lower-level, and the deployment of UAVs through UDG method in the upper-level, similar to TDE. Note that we utilize the same data and parameter settings as those of the DDPG scheme in order to ensure the effectiveness of the comparison.
- No Collaboration: In order to verify the effectiveness of the proposed TDE and UCC system, we compare the performance of multi-UAV edge computing systems without collaborative computing.

1) Uniform Distribution of Users: We first compare TDE with the above three schemes in terms of the uniform distribution of users, as shown in Table V. Observations from Table V reveal that, first, for the ToDeTaS scheme, which is based on the benchmark DE, the population size is unstable and the population quality is poor due to the random initialization of the ToDeTaS population. Additionally, task scheduling based on greedy algorithms can result in getting trapped in local optima. Therefore, the optimized results of ToDeTaS are not satisfactory; the average task completion rate is unstable, and the average energy consumption is too high. When deployed on a larger user scale, such as $J = 1000$, the ToDeTaS achieves an energy consumption of 52.1k. In comparison, the TDE algorithm has a reduction in energy consumption of approximately 32.4%.

Second, regarding the DDPG scheme, the deterministic policy of DDPG restricts the thorough exploration of the state space and action space. Additionally, the issue of overestimating Q-values results in less robust optimization. Consequently, the performance of offloading decisions and resource allocation at the lower level is very erratic. Although DDPG can achieve a lower average energy consumption, the average task completion rate is not optimal. When deployed on a larger user scale, such as $J = 1000$, DDPG's NC is merely 916.

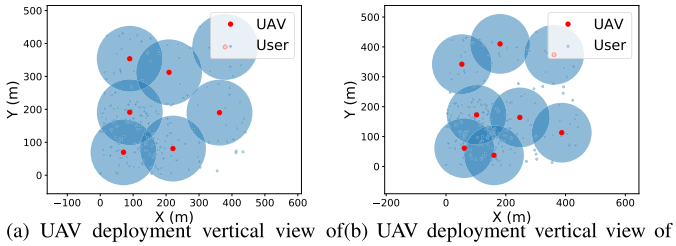
Third, within the No Collaboration scheme, the computation resources of individual UAVs are limited due to the absence of collaborative computing. Consequently, UAVs covering areas with dense user populations become overloaded. Therefore, as the number of users increases, the No Collaboration scheme

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.



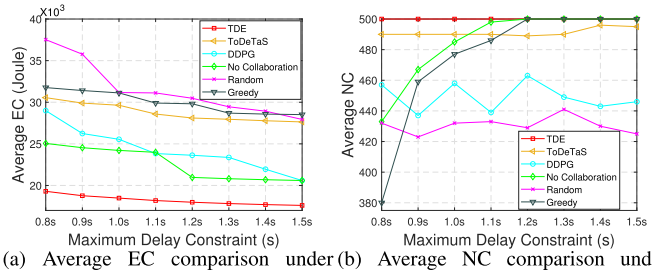
(a) Iteration comparison of EC for $J = 400$ (b) Iteration comparison of NC for $J = 400$

Fig. 5. Comparison of iteration performance.



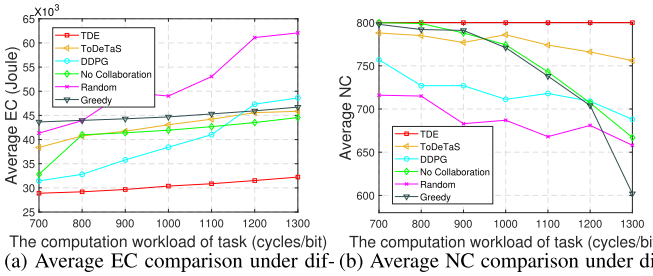
(a) UAV deployment vertical view of TDE (b) UAV deployment vertical view of ToDeTaS

Fig. 6. Comparison of UAV deployment at $J = 200$ uneven distribution.



(a) Average EC comparison under different maximum delay constraint $T_j^{\max}$ for $J = 500$. (b) Average NC comparison under different maximum delay constraint $T_j^{\max}$ for $J = 500$.

Fig. 7. Comparison of TDE in different delay constraint.



(a) Average EC comparison under different task computation workload C_j for $J = 800$. (b) Average NC comparison under different task computation workload C_j for $J = 800$.

Fig. 8. Comparison of TDE in different task computation workload.

results in high average energy consumption and a small average number of completed tasks. In comparison, TDE achieves optimal performance with low average energy consumption and high task completion rates on average.

In addition, we present the vertical view of UAV deployment for TDE and ToDeTaS at $J = 200$ with a uniform distribution as shown in Fig. 3. This demonstrates that TDE utilizes fewer UAVs to achieve a more superior and reasonable deployment scheme compared to ToDeTaS. It should be noted that some users are not covered because they possess sufficient resources to compute independently, and TDE decides against covering these users in order to enhance the overall energy efficiency of the UCC system.

Moreover, the iteration performance of the average energy consumption compared to ToDeTaS and DDPG in the case of $J = 400$ is presented in Fig. 5(a). The same data and parameter settings are provided to the three schemes in order to ensure the effectiveness of the comparison. As shown in Fig. 5(a), the TDE converges fast and achieves the lowest EC. Given that TDE leverages the enhanced differential evolution algorithm, UDG, which facilitates the generation of high-quality populations, the TDE is poised to commence iterations from an advantageous initial outcome. With the incorporation of the target smoothing strategy inherent in the TD3 at the lower-level, a stable convergence towards minimal energy consumption is ensured.

Furthermore, we compare the NC performance of the three schemes as shown in Fig. 5(b). It can be observed that DDPG exhibits significant fluctuation. This instability in DDPG can be attributed to approximation errors in the value function, coupled with an overestimation bias in action-value estimates. As the iteration progresses, ToDeTaS stabilizes gradually and ultimately achieves a 100% task completion rate, whereas TDE maintains perfect NC performance. Clearly, the performance of TDE is optimal in both the iterative process and the optimization result.

2) Uneven Distribution of Users: Then, we compare these schemes in more stringent and challenging real-world scenarios with uneven user distribution. As illustrated in Fig. 6, half of the users are concentrated in the bottom-left corner of the square area. The comparison results, shown in Table VI, indicate that TDE maintains optimal performance. Notably, the average number of completed tasks within the No Collaboration scheme decreases significantly as the number of users increases. When deployed on a larger user scale, i.e., $J = 1000$, the energy consumption of the No Collaboration scheme is 47.9k, whereas for NC it is only 579, with nearly half of the tasks unable to be completed.

This decrease is attributed to the limited resources of individual UAVs facing an ever-increasing density of users. Moreover, UAVs covering areas with scattered users tend to waste a considerable amount of idle resources, leading to a low task completion rate and high energy consumption. This highlights that the challenges posed by the uneven distribution of users are severe and underscores the effectiveness of UAV collaborative computing. In contrast, the TDE algorithm achieved a nearly 30% average improvement in task completion rate when dealing with uneven distribution of users.

Moreover, the vertical view of UAV deployment for TDE and ToDeTaS, at $J = 200$ with uneven distribution, is depicted in Fig. 6. It can be observed that a large number of users are concentrated in a smaller area due to terrain factors or events, and our TDE scheme utilizes fewer UAVs to achieve a more superior and reasonable deployment scheme compared to ToDeTaS. It's noteworthy that the energy consumption of TDE is reduced compared to scenarios with a uniform distribution

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
COMPARISON OF SCHEMES UNDER UNEVEN DISTRIBUTION OF USERS IN TERMS OF AVERAGE EC (JOULE) AND AVERAGE NC

Uneven J	ToDeTaS		DDPG		No Collaboration		TDE	
	Average EC	Average NC	Average EC	Average NC	Average EC	Average NC	Average EC	Average NC
100	4235.79	100	4720.50	93	**5205.03**	**86**	**3186.13**	**100**
200	8614.95	200	8572.38	187	10495.18	151	**7408.81**	**200**
300	16837.48	298	14413.52	270	14801.21	186	**10298.72**	**300**
400	22190.81	398	19859.60	365	21116.43	276	**14881.05**	**400**
500	24789.53	493	21763.76	454	22500.02	312	**17660.63**	**500**
600	30800.60	576	27221.72	538	27737.13	344	**21633.28**	**600**
700	36448.38	689	33243.16	630	33037.34	414	**24350.04**	**700**
800	41623.16	783	38675.02	727	38314.98	478	**26697.08**	**800**
900	46267.25	881	44391.26	807	42606.71	542	**29154.30**	**900**
1000	**50122.41**	973	46417.73	890	47944.22	**579**	**35054.85**	**1000**

TABLE VII
EXPERIMENTAL RESULTS OF TDE-L, TDE-U, TDE-R, AND TDE IN TERMS OF AVERAGE NC

J	Average NC			
	TDE-L	TDE-U	TDE-R	TDE
100	71	95	82	100
200	140	190	171	200
300	214	278	260	300
400	286	381	345	400
500	353	479	432	500
600	434	578	524	600
700	505	675	617	700
800	576	780	684	800
900	665	882	780	900
1000	726	975	876	1000

of users, as the users are more closely clustered, allowing an individual UAV to cover more users. Consequently, the number of UAVs can be reasonably reduced, thereby enhancing system energy efficiency and resource utilization. In summary, TDE achieves optimal performance in scenarios of both uniform and uneven user distributions, demonstrating its high robustness and adaptability to real-world conditions.

3) Comparison With Benchmark Schemes: In this subsection, we compare TDE with three benchmarks, i.e. TDE-L, TDE-U, and TDE-R, to verify the effectiveness of the UCC system and TDE. For TDE-L, all tasks can only be executed locally. For TDE-U, all tasks can only be executed on UAVs. For TDE-R, all tasks are randomly selected to execute locally or on the UAV. As shown in Table VII, the NC of TDE-L is lowest due to the limited resources of the user device. In addition, TDE-U cannot guarantee the completion of all tasks, due to the fact that some users may not be covered by any UAV. In principle, TDE can alleviate the limitation of the three benchmarks. This comparison confirms the effectiveness of the proposed UCC system and TDE.

C. Performance of the TDE in Different Circumstances

To further demonstrate the effectiveness of the TDE, we perform comparative experiments with all other schemes under different circumstances in this section. It is worth noting that to enhance the thoroughness of the experiments, we introduce a Greedy scheme and a Random scheme for comparison. The Greedy scheme offloads all tasks to the UAV with the greatest computational resources, while the Random scheme offloads tasks randomly, either locally or to various UAVs.

Firstly, we constructed comparative experiments to study the impact of delay constraints on TDE in the scenario of $J = 500$, as shown in Fig. 7. It can be seen from Fig. 7(a) that as the delay tolerance increases, the energy consumption of TDE and other schemes decreases. This is attributed to the fact that the task difficulty decreases as the delay tolerance increases, and thus the required resources and energy consumption decrease. Due to referenceless giving the current best choice, the EC of the Greedy scheme does not decrease significantly and falls into a local optimal solution. In contrast, the EC of TDE is the lowest under all delay constraints. Moreover, As given in Fig. 7(b), when the delay tolerance is small, the task of Greedy scheme is difficult to complete due to the strict requirements. The task difficulty decreases as the delay tolerance increases, and thus the NC of No Collaboration scheme and Greedy scheme gradually increase and finally achieve 100% task completion rate. DDPG and Random schemes oscillate on a poor NC and ToDeTaS oscillates on a slightly better NC. In contrast, the TDE obtains the optimal performance of NC under all delay constraints and maintains 100% task completion rate.

Lastly, the comparative experiments on the impact of task computation workload on TDE in the scenario of $J = 800$ are presented in Fig. 8. It can be observed from Fig. 8(a) that as the task computation workload increases (task computation difficulty becomes higher), all other schemes show a significant upward trend. However, TDE maintains the lowest energy consumption despite a gradual increase. Furthermore, as shown in Fig. 8(b), other schemes experience continuous task failures as the task computation amount increases. The decline of the Greedy scheme is abrupt, resulting from the referenceless selection of drones with the most abundant resources for task computation, which leads to a decrease in the overall resource utilization rate and, consequently, a sharp drop in the task completion rate. Meanwhile, TDE is always able to maintain the completion of all tasks. This demonstrates the robustness and effectiveness of the TDE.

V. DISCUSSION

A. Potential Challenges

The practical deployment of multi-UAV systems could encounter some potential challenges. Firstly, different regions may have their own regulations and guidelines for the use of UAVs, making the proposed system deployment have to

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

comply with these laws. Although the proposed system framework TDE can adjust the optimization of altitude and distance according to the specific legal regulations, merely optimizing the algorithm's height is not sufficient and we should also consider meeting the minimum distance requirements for airports, private properties, and prohibited flying zones. Secondly, with the increasing prevalence of UAVs, the risk of signal interference is also on the rise. How to achieve efficient interference management during task offloading and data transmission play an imperative role in improving the effectiveness of multi-UAV system. Additionally, the complexity of UAV coordination should not be underestimated. The cooperation among UAVs face the challenge of heterogeneity of UAVs from different manufacturers in terms of model, hardware, and architecture.

B. Future Research

The future research directions for our framework TDE have the following several aspects. 1) The proposed system will be directly integrated with the existing telecommunications infrastructure, treating flying edge servers as static ones, thus having excellent scalability and the flexibility. 2) We will explore the enhancement of the integrated system's architecture to support an increasing number of UAVs and process large-scale operations, thereby boosting the coverage of our UAV system. 3) We will further study the robust algorithms that can be adaptively adjusted by unstable network conditions and various user demands. This is particularly crucial in scenarios such as disaster recovery and sudden public events. 4) We also plan to utilize artificial intelligence to predict demands and adjust system parameters, thereby maintaining performance during peak and emergency situations, and improving the intelligence of the system.

VI. Conclusion

In this paper, we propose a bi-level optimization framework named TDE to maximize the energy-efficiency of the multi-UAV Collaborative edge Computing (UCC) system by jointly optimizing the deployment of UAVs and the task collaborative offloading decisions as well as computation and communication resource allocation, which is NP-hard MINLP. The UAVs collaborate to handle delay-sensitive tasks of users. We transform the original MINLP into a bi-level optimization problem. We modify the differential evolution algorithm to form a new UDG algorithm that optimizes the deployment of UAVs in the upper-level, and propose the offloading decision and resource allocation algorithm based on TD3 scheme in the lower-level. Experimental results demonstrate the effectiveness and superiority of multi-UAV collaborative computing, and the proposed TDE achieves higher performance compared to existing works such as DDPG, ToDeTaS, and other benchmark schemes in the terms of energy-efficiency and task completion rate under various network parameter settings.

References

[1] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, 4th Quart., 2021.

[2] P. Yu et al., "Energy-efficient coverage and capacity enhancement with intelligent UAV-BSs deployment in 6G edge networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7664–7675, Jul. 2023.

[3] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, 3rd Quart., 2019.

[4] F. S. Abkenar et al., "A survey on mobility of edge computing networks in IoT: State-of-the-art, architectures, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2329–2365, 4th Quart., 2022.

[5] L. Qu, G. Xu, Z. Zeng, N. Zhang, and Q. Zhang, "UAV-assisted RF/FSO relay system for space-air-ground integrated network: A performance analysis," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6211–6225, Aug. 2022.

[6] S. Han, K. Zhu, M. Zhou, and X. Liu, "Joint deployment optimization and flight trajectory planning for UAV assisted IoT data collection: A bilevel optimization approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21492–21504, Nov. 2022.

[7] X. Zhu and M. Zhou, "Maximal weighted coverage deployment of UAV-enabled rechargeable visual sensor networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 11293–11307, Oct. 2023.

[8] R. Han, Y. Wen, L. Bai, J. Liu, and J. Choi, "Age of information aware UAV deployment for intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2705–2715, Mar. 2022.

[9] W. Feng et al., "Hybrid beamforming design and resource allocation for UAV-aided wireless-powered mobile edge computing networks with NOMA," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3271–3286, Nov. 2021.

[10] X. Zhang and L. Duan, "Fast deployment of UAV networks for optimal wireless coverage," *IEEE Trans. Mobile Comput.*, vol. 18, no. 3, pp. 588–601, Mar. 2019.

[11] W. Xu et al., "Minimizing the deployment cost of UAVs for delay-sensitive data collection in IoT networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 812–825, Apr. 2022.

[12] N. Lin, Y. Liu, L. Zhao, D. O. Wu, and Y. Wang, "An adaptive UAV deployment scheme for emergency networking," *IEEE Trans. Wireless Commun.*, vol. 21, no. 4, pp. 2383–2398, Apr. 2022.

[13] C. Han, A. Liu, K. An, G. Zheng, and X. Tong, "Distributed UAV deployment in hostile environment: A game-theoretic approach," *IEEE Wireless Commun. Lett.*, vol. 11, no. 1, pp. 126–130, Jan. 2022.

[14] J. Su, S. Yu, B. Li, and Y. Ye, "Distributed and collective intelligence for computation offloading in aerial edge networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7516–7526, Jul. 2023.

[15] L. Yang, H. Yao, J. Wang, C. Jiang, A. Benslimane, and Y. Liu, "Multi-UAV-enabled load-balance mobile-edge computing for IoT networks," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6898–6908, Aug. 2020.

[16] J. Luo, J. Song, F.-C. Zheng, L. Gao, and T. Wang, "User-centric UAV deployment and content placement in cache-enabled multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5656–5660, May 2022.

[17] X. Li, H. Yao, J. Wang, X. Xu, C. Jiang, and L. Hanzo, "A near-optimal UAV-aided radio coverage strategy for dense urban areas," *IEEE Trans. Veh. Technol.*, vol. 68, no. 9, pp. 9098–9109, Sep. 2019.

[18] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1960–1972, Mar. 2022.

[19] Y. Wang et al., "Task offloading for post-disaster rescue in unmanned aerial vehicles networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1525–1539, Aug. 2022.

[20] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Apr. 2021.

[21] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Apr. 2019.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.

[22] F. Tütüncüoglu, S. Jošilo, and G. Dán, "Online learning for rate-adaptive task offloading under latency constraints in serverless edge computing," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 695–709, Apr. 2023.

[23] Y. Li, S. Xu, Y. Wu, and D. Li, "Network energy-efficiency maximization in UAV-enabled air–ground-integrated deployment," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13209–13222, Aug. 2022.

[24] Y. Wang, Z.-Y. Ru, K. Wang, and P.-Q. Huang, "Joint deployment and task scheduling optimization for large-scale mobile users in multi-UAV-enabled mobile edge computing," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3984–3997, Sep. 2020.

[25] L. Zhang, F. Wen, and X. Li, "UAVs-assisted edge networks optimization based on constrained particle swarm algorithm," in *Proc. IEEE 23rd Int. Conf. Commun. Technol. (ICCT)*, Oct. 2023, pp. 1699–1704.

[26] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.

[27] Z. Dai, Y. Zhang, W. Zhang, X. Luo, and Z. He, "A multi-agent collaborative environment learning method for UAV deployment and resource allocation," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 8, pp. 120–130, 2022.

[28] Y. Nie, J. Zhao, F. Gao, and F. R. Yu, "Semi-distributed resource management in UAV-aided MEC systems: A multi-agent federated reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13162–13173, Dec. 2021.

[29] P. Luong, F. Gagnon, L. Tran, and F. Labeau, "Deep reinforcement learning-based resource allocation in cooperative UAV-assisted wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7610–7625, Nov. 2021.

[30] T. Do-Duy, L. D. Nguyen, T. Q. Duong, S. R. Khosravirad, and H. Claussen, "Joint optimisation of real-time deployment and resource allocation for UAV-aided disaster emergency communications," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3411–3424, Nov. 2021.

[31] S. Arzykulov, A. Celik, G. Nauryzbayev, and A. M. Eltawil, "UAV-assisted cooperative & cognitive NOMA: Deployment, clustering, and resource allocation," *IEEE Trans. Cognit. Commun. Netw.*, vol. 8, no. 1, pp. 263–281, Mar. 2022.

[32] X. Pang et al., "Uplink precoding optimization for NOMA cellular-connected UAV networks," *IEEE Trans. Commun.*, vol. 68, no. 2, pp. 1271–1283, Feb. 2020.

[33] N. Zhao et al., "Joint trajectory and precoding optimization for UAV-assisted NOMA networks," *IEEE Trans. Commun.*, vol. 67, no. 5, pp. 3723–3735, May 2019.

[34] J. Wang, Z. Jiao, J. Chen, X. Hou, T. Yang, and D. Lan, "Blockchain-aided secure access control for UAV computing networks," *IEEE Trans. Netw. Sci. Eng.*, early access, Oct. 16, 2023, doi: 10.1109/TNSE.2023.3324639.

[35] L. Tan, Z. Kuang, J. Gao, and L. Zhao, "Energy-efficient collaborative multi-access edge computing via deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7689–7699, Jun. 2023.

[36] Y. Dong, S. Guo, J. Liu, and Y. Yang, "Energy-efficient fair cooperation fog computing in mobile edge networks for smart city," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7543–7554, Oct. 2019.

[37] Z. Kuang, G. Liu, G. Li, and X. Deng, "Energy efficient resource allocation algorithm in energy harvesting-based D2D heterogeneous networks," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 557–567, Feb. 2019.

[38] C. Lin et al., "Maximizing energy efficiency of period-area coverage with a UAV for wireless rechargeable sensor networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 4, pp. 1657–1673, Aug. 2023.

[39] L. Ji and S. Guo, "Energy-efficient cooperative resource allocation in wireless powered mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4744–4754, Jun. 2019.

[40] J. Liu, S. Guo, K. Liu, and L. Feng, "Resource provision and allocation based on microeconomic theory in mobile edge computing," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1512–1525, May/Jun. 2022.

[41] N. Nouri, J. Abouei, A. R. Sepasian, M. Jaseemuddin, A. Anpalagan, and K. N. Plataniotis, "Three-dimensional multi-UAV placement and resource allocation for energy-efficient IoT communication," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 2134–2152, Feb. 2022.

[42] H. Zhao, H. Wang, W. Wu, and J. Wei, "Deployment algorithms for UAV airborne networks toward on-demand coverage," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 2015–2031, Sep. 2018.

[43] J. Ren et al., "Collaborative cloud and edge computing for latency minimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 5031–5044, May 2019.

[44] E. M. Mohamed, "Deployment of mmWave multi-UAV mounted RISs using budget constraint Thompson sampling with collision avoidance," *ICT Exp.*, vol. 10, no. 2, pp. 277–284, Apr. 2024.

[45] N. H. Awad, M. Z. Ali, and P. N. Suganthan, "Ensemble of parameters in a sinusoidal differential evolution with niching-based population reduction," *Swarm Evol. Comput.*, vol. 39, pp. 141–156, Apr. 2018.

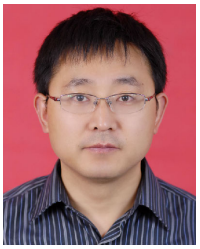
[46] B. Hutt and K. Warwick, "Synapsing variable-length crossover: Meaningful crossover for variable-length genomes," *IEEE Trans. Evol. Comput.*, vol. 11, no. 1, pp. 118–131, Feb. 2007.

[47] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.

[48] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.



Lin Tan (Student Member, IEEE) received the M.Sc. degree in computer technology from the Central South University of Forestry and Technology, Changsha, China, in 2022. He is currently pursuing the Ph.D. degree in computer science and technology with Chongqing University, Chongqing, China. His current research interests include edge computing, deep reinforcement learning, and optimization algorithm and its application. He has received the Outstanding Master's Thesis Award from Hunan Computer Federation.



Songtao Guo (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in computer software and theory from Chongqing University, Chongqing, China, in 1999, 2003, and 2008, respectively. From 2010 to 2011, he was a Senior Research Associate with the City University of Hong Kong. From May 2011 to May 2012, he was a Visiting Scholar with Stony Brook University, Stony Brook, NY, USA. From 2011 to 2012, he was a Professor with Chongqing University. From 2013 to 2018, he was a Professor with Southwest University. He is currently a Full Professor with Chongqing University. He has received many research grants as a Principal Investigator from the National Science Foundation of China and Chongqing and the Postdoctoral Science Foundation of China. He has published more than 100 scientific papers in leading refereed journals and conferences. His research interests include wireless sensor networks, wireless ad hoc networks, and mobile edge computing.



Pengzhan Zhou (Member, IEEE) received the B.S. degree in applied physics and applied mathematics from Shanghai Jiao Tong University, Shanghai, China, and the Ph.D. degree in electrical engineering from Stony Brook University, Stony Brook, NY, USA. He is currently a Professor with the College of Computer Science, Chongqing University, Chongqing, China. His research interests include the Internet of Things, reinforcement learning, federated learning, and incentivizing mechanism.

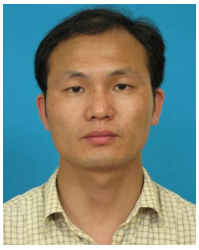
Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.



Zhufang Kuang (Member, IEEE) received the M.Sc. degree in computer science from the National University of Defense Technology in 2006 and the Ph.D. degree in computer science from Central South University, Changsha, China, in 2012. He was a Post-Doctoral Researcher with the School of Software, Central South University. From 2015 to 2016, he was a Visiting Scholar/Professor with the University of Victoria, Victoria, BC, Canada. He is currently a Full Professor with the Department of Computer Science, Central South University of Forestry and Technology. His current research interests include wireless communications and networking, the Internet of Things (IoT), mobile edge computing, and artificial intelligence. He is a Distinguished Member of CCF, a member of the CCF Internet of Things Council, and a member of the CCF Network and Data Communications Council, and ACM. From 2022 to 2023, he was the Chair of CCF YOCSEF Changsha.



Saiqin Long received the Ph.D. degree in computer applications technology from the South China University of Technology, Guangzhou, China, in 2014. She is currently a Professor with the College of Information Science and Technology, Jinan University, China. Her research interests include cloud computing, edge computing, parallel and distributed systems, and the Internet of Things. She has published more than 20 refereed articles in these areas, most of which are published in premium conferences and journals, including IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, and IEEE TRANSACTIONS ON MOBILE COMPUTING. She is a member of the Chinese Computer Federation (CCF).



Zhetao Li (Senior Member, IEEE) received the B.Eng. degree in electrical information engineering from Xiangtan University in 2002, the M.Eng. degree in pattern recognition and intelligent system from Beihang University in 2005, and the Ph.D. degree in computer application technology from Hunan University in 2010. From December 2013 to December 2014, he was a Post-Doctoral Researcher in wireless networks with Stony Brook University. From December 2014 to December 2015, he was an Invited Professor with Ajou University. He is currently a Professor with the College of Information Engineering, Xiangtan University, and a Professor with the College of Information Science and Technology, Jinan University, China. He has published about 30 papers and submitted about 40 patents on wireless networks and signal processing in the past five years. His research interests include mobile computing, wireless communications, and signal processing. He was the lead guest editor of two special issues of PPAN.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:12:05 UTC from IEEE Xplore. Restrictions apply.